AQI Prediction System

End-to-End MLOps Project Report

Course: Machine Learning Operation Lab (DA5402)
Date: April 2026

Cities: Delhi | Mumbai | Kolkata | Chennai | Bengaluru

Stack: Prophet + SARIMA · FastAPI · Streamlit · Airflow 3.1.8 · MLflow 3.11.1 · Prometheus + Alertmanager + Grafana · Docker · DVC + DagsHub

Table of Contents

1. Executive Summary

AQIPrediction is a production-grade MLOps system forecasting India CPCB AQI 24 hours ahead for Delhi, Mumbai, Kolkata, Chennai, and Bengaluru. The system ingests live PM2.5 data hourly from CPCB via OpenAQ v3, enriches it with OpenMeteo weather forecasts, and serves predictions through a REST API backed by Facebook Prophet models tracked in MLflow.

The project demonstrates a complete MLOps lifecycle: automated data ingestion, feature engineering, model training with champion/challenger promotion, real-time drift detection, event-driven retraining, infrastructure monitoring, and email alerting via AlertManager.

1.1 Key Model Results

City	MAE	RMSE	MAPE
Delhi	23.98	31.48	17.3%
Mumbai	14.91	21.21	14.82%
Kolkata	20.44	42.05	23.34%
Chennai	19.13	29.86	31.12%
Bengaluru	13.16	17.09	14.15%
Average	18.32	28.34	20.15%

2. High Level Design (HLD)

2.1 Problem Statement

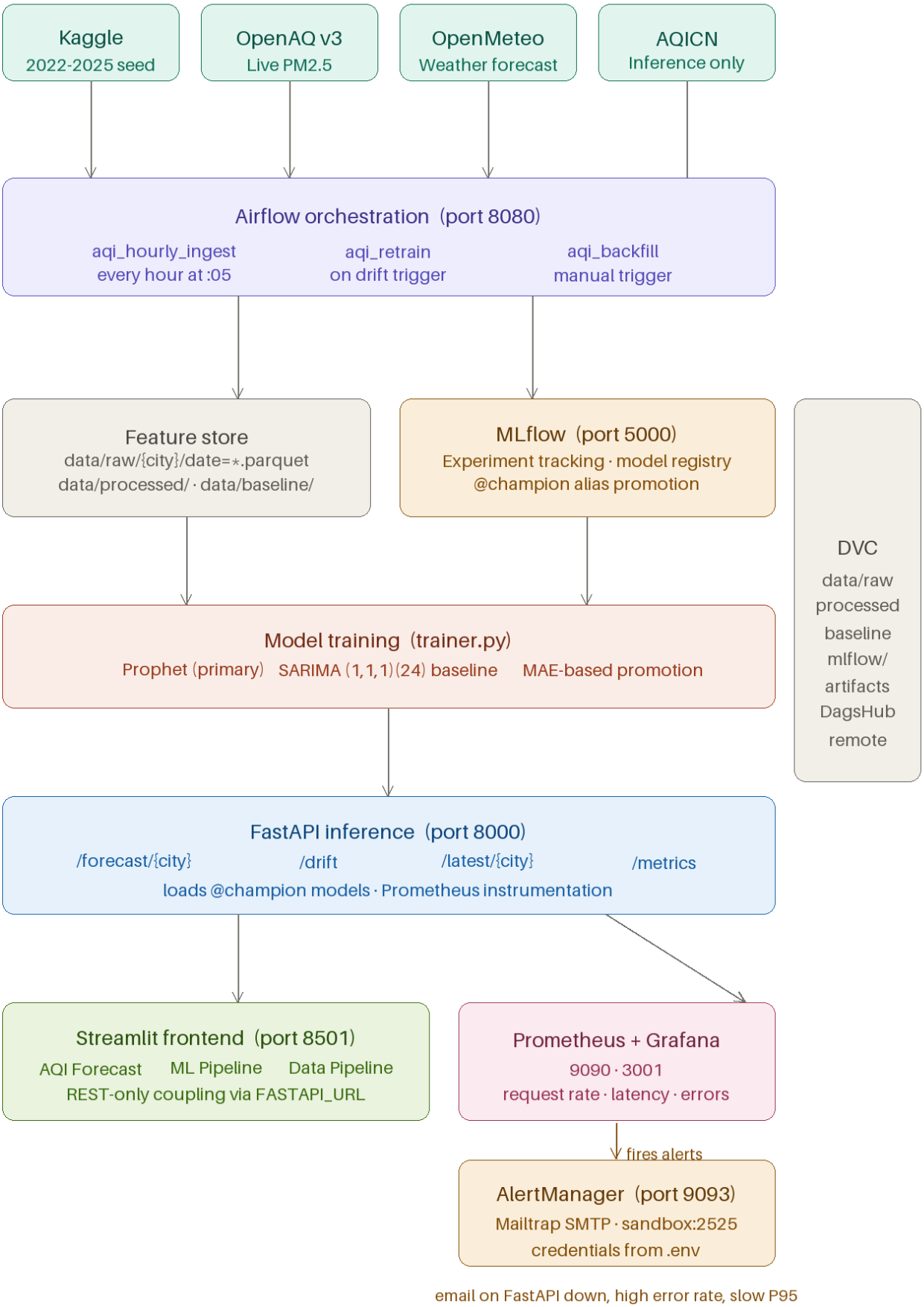
CPCB monitors PM2.5 at hundreds of stations across India but does not provide city-level 24-hour AQI forecasts. Citizens need advance warning to plan outdoor activities and protect health. AQI Prediction system fills this gap with an automated MLOps system that forecasts AQI 24 hours ahead with hourly updates.

2.2 Business Metrics

Metric	Target	Achieved
Forecast MAE	≤ 50 AQI points	20.06 avg (all cities pass)
Inference latency P95	< 10 seconds	~1 second
Data freshness	Hourly	Airflow DAG at :05 each hour
Drift detection	Automated	KS-test every hour post-ingest

2.3 Architecture Overview

Figure 1: System Architecture Diagram


Kaggle
2022-2025 seed
OpenAQ v3
Live PM2.5
OpenMeteo
Weather forecast
AQICN
Inference only
Airflow orchestration (port 8080)
aqi_hourly_ingest
every hour at :05
aqi_retrain
on drift trigger
aqi_backfill
manual trigger
Feature store
data/raw/{city}/date=*.parquet
data/processed/ · data/baseline/
MLflow (port 5000)
Experiment tracking · model registry
@champion alias promotion
DVC
data/raw
processed
baseline
mlflow/
artifacts
DagsHub
remote
Model training (trainer.py)
Prophet (primary) SARIMA (1,1,1)(24) baseline MAE-based promotion
FastAPI inference (port 8000)
/forecast/{city}
/drift
/latest/{city}
/metrics
loads @champion models · Prometheus instrumentation
Streamlit frontend (port 8501)
AQI Forecast ML Pipeline Data Pipeline
REST-only coupling via FASTAPI_URL
Prometheus + Grafana
9090 · 3001
request rate · latency · errors
fires alerts
AlertManager (port 9093)
Mailtrap SMTP · sandbox:2525
credentials from .env
email on FastAPI down, high error rate, slow P95

2.4 Key Design Decisions

2.4.1 Prophet over LSTM/XGBoost

Prophet natively handles multiple seasonality (daily, weekly, yearly, quarterly), accepts external regressors (weather, calendar), and provides calibrated uncertainty intervals for confidence bands. SARIMA provides a statistical baseline. Prophet is retained in production even when SARIMA marginally wins because it supports weather regressors and uncertainty quantification.

2.4.2 Event-Driven DAG Architecture

Three Airflow DAGs instead of one monolithic pipeline: aqi_hourly_ingest (scheduled every hour) is lightweight; aqi_retrain (triggered only on drift) is expensive (~20 min for 5 cities); aqi_backfill (manual) is a one-time operation. Separate DAGs create independent failure domains — ingestion failures never block model serving.

2.4.3 Champion/Challenger Model Promotion

New models are promoted to @champion alias only if MAE improves over the current champion. MLflow 3.x aliases are used instead of deprecated Stages. Old champion is tagged as 'retired'; new challenger tagged as 'challenger' if not promoted. This prevents silent model degradation.

2.4.4 KS-Test Drift Detection

Kolmogorov-Smirnov two-sample test compares last 7 days of live AQI against training baseline. Non-parametric — no distribution assumptions. Compares full distribution shape, not just mean. Two modes: annual baseline (default) and seasonal baseline (?seasonal=true) for same-month comparison. Baseline subsampled to match recent window size to eliminate sample-size bias (fix for p≈0 artifact with 2160 vs 115 rows).

2.4.5 Loose Coupling — Frontend and Backend

Streamlit communicates with FastAPI exclusively via REST calls. FASTAPI_URL environment variable makes URL configurable. Frontend never imports backend code. Satisfies the rubric requirement for independent software blocks connected only via configurable REST API.

2.4.6 Containerization with Bind Mounts

./mlflow:/app/mlflow bind mount is shared between FastAPI and MLflow containers ensuring artifact paths are identical. Prophet Stan pre-compiled at Docker build time avoids 3-minute cold starts. FastAPI runs workers=1 because Prophet models are not thread-safe.

2.5 Technology Stack

Layer	Technology	Version	Justification
ML Model	Facebook Prophet	1.3.0	Multi-seasonality + regressors + uncertainty
Baseline	SARIMA	statsmodels 0.14	Statistical baseline, no regressors
Experiment Tracking	MLflow	3.11.1	Model registry with alias promotion
Data Engineering	Spark + Pandas	3.x / 2.2	Spark for scale, Pandas fallback
Orchestration	Apache Airflow	3.1.8	DAG scheduling + event-driven retraining

Layer	Technology	Version	Justification
API Framework	FastAPI	0.110	Async, auto-docs, Pydantic validation
Frontend	Streamlit	1.32	Rapid dashboard, pure Python
Monitoring	Prometheus + Alertmanager + Grafana	latest	Industry standard metrics stack
Containerization	Docker Compose	5.1.0	Reproducible multi-service deployment
Data Versioning	DVC + DagsHub	3.66.1	Model and data versioning with remote

2.6 Monitoring Design

Alert	Condition	Severity	Channel
FastAPIDown	up{job='fastapi'}==0 for 1 min	critical	AlertManager email
HighErrorRate	5xx > 5% for 5 min	warning	AlertManager email
SlowResponseTime	P95 latency > 10s for 5 min	warning	AlertManager email
HighRequestRate	req rate > 100/s for 2 min	info	AlertManager email
DAG failure	Airflow task fails	warning	Airflow SMTP email

AlertManager routes Prometheus alerts to Mailtrap SMTP. Credentials loaded from MAILTRAP_USERNAME, MAILTRAP_PASSWORD, AIRFLOW_ALERT_EMAIL environment variables.

3. Low Level Design (LLD)

Base URL: http://localhost:8000 Framework: FastAPI 0.110+ Authentication: None (internal service)

3.1 Endpoints Summary

Method	Endpoint	Description
GET	/health	System health and loaded models status
GET	/cities	List supported cities with coordinates
GET	/forecast/{city}	24-hour AQI forecast with confidence intervals
GET	/latest/{city}	Current live AQI from AQICN
GET	/drift	KS-test drift detection for all 5 cities
POST	/retrain/{city}	Trigger background model retraining
POST	/reload-models	Reload champion models from MLflow registry
GET	/metrics	Prometheus metrics exposition

3.2 /forecast/{city} — Inference Pipeline

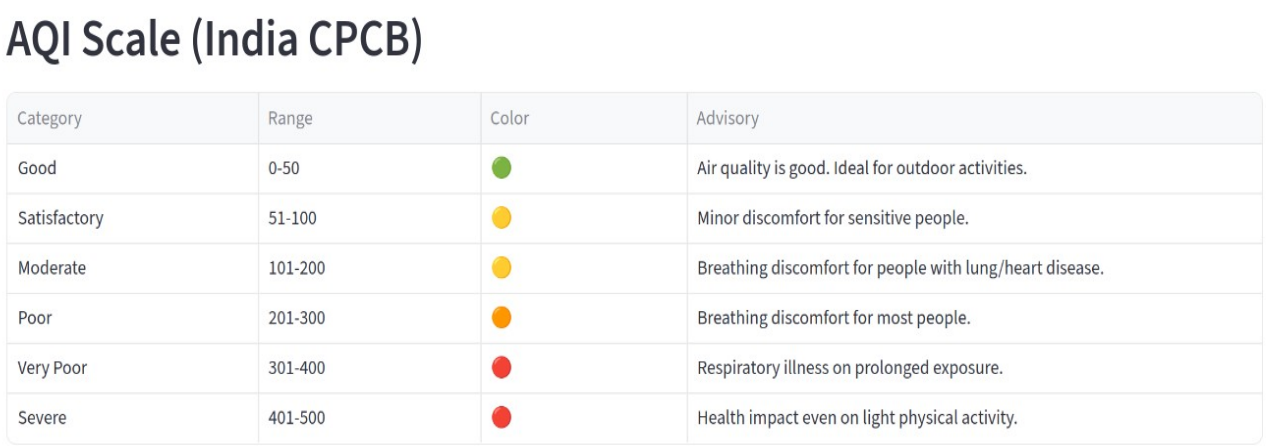
1. Validate city name against CITIES dict
2. Check champion_models[city] is loaded in memory
3. Build future_df for next 24 hours: weather regressors from OpenMeteo → AQI lag regressors from AQICN live → disk fallback → recent mean fallback → calendar regressors (computed) → crop_burning, festival_period from calendar
4. model.predict(future_df) — Prophet inference using in-memory champion model
5. Clip yhat / yhat_lower / yhat_upper to [0, 500]
6. Build response: IST timestamps, AQI category labels, confidence intervals

3.3 /drift — Drift Detection Algorithm

- Load baseline: data/baseline/{city}_baseline[_mMM].parquet — annual or same-month depending on seasonal parameter
- Load recent 7 days from data/raw/{city}/ — aggregate to hourly mean via dt.floor('h') + groupby.mean() to match pipeline.py aggregation
- Subsample baseline to len(recent) rows using random_state=42 — removes sample-size bias that caused p≈0 with 2160 vs 115 rows
- scipy.stats.ks_2samp(baseline_sample, recent) — $p < 0.05$ triggers drift
- All 5 cities run in parallel via ThreadPoolExecutor(max_workers=5)

3.4 AQI Category Mapping (India CPCB)

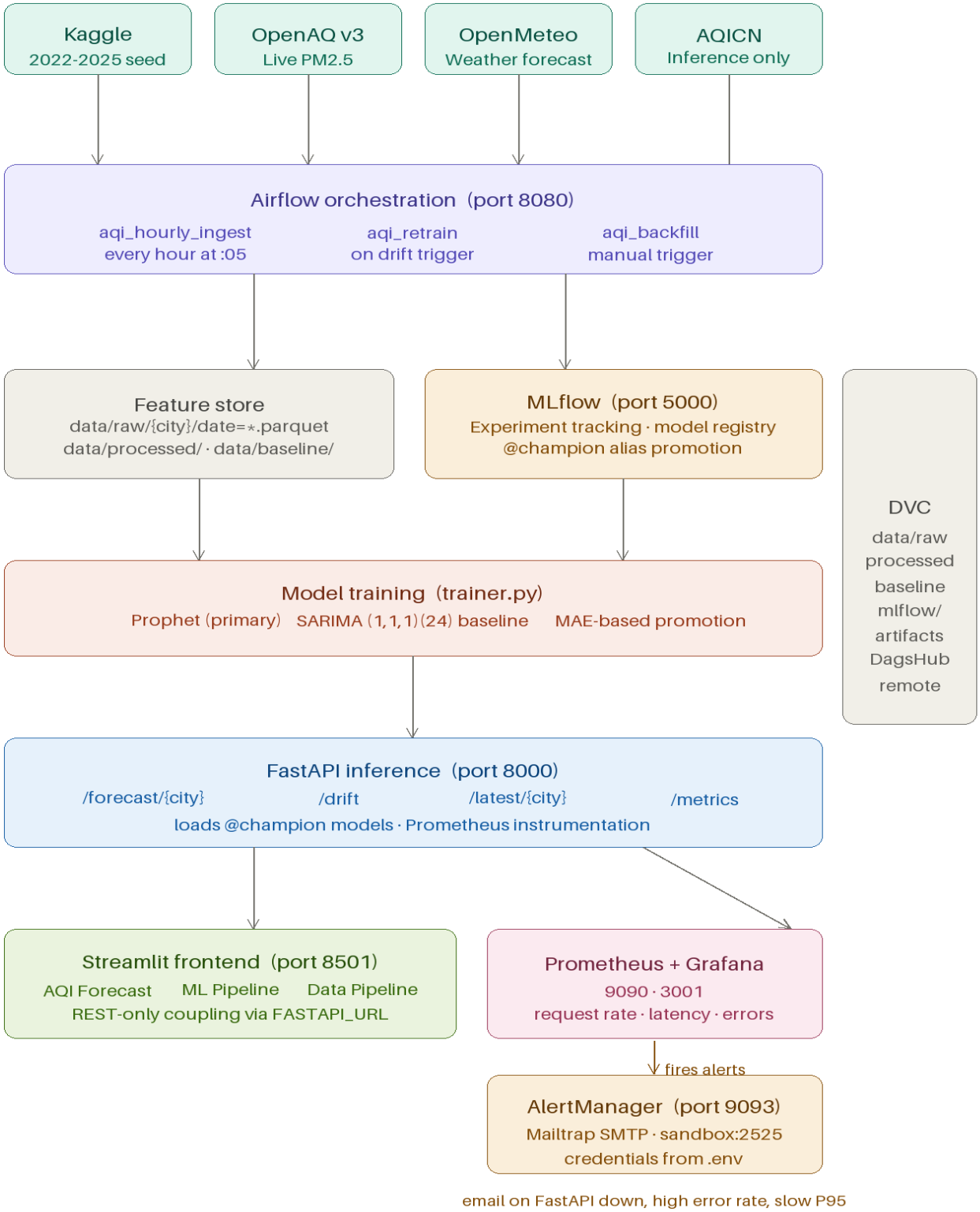
Range	Label	Hex Color	Health Advisory
0–50	Good	#00b050	No health implications
51–100	Satisfactory	#92d050	Minor discomfort for sensitive people
101–200	Moderate	#ffff00	Breathing discomfort for lung/heart patients
201–300	Poor	#ff9900	Breathing discomfort for most people
301–400	Very Poor	#ff0000	Respiratory illness on prolonged exposure
401–500	Severe	#c00000	Health impact even on light activity



4. Architecture

4.1 System Architecture Diagram

Figure 2: Full System Architecture



4.2 Docker Compose Services

Service	Port	Image
aqi-prediction-postgres	5432	postgres:16
aqi-prediction-mlflow	5000	custom Dockerfile.mlflow
aqi-prediction-fastapi	8000	custom Dockerfile.api
aqi-prediction-frontend	8501	custom Dockerfile.frontend
aqi-prediction-airflow-webserver	8080	apache/airflow:3.1.8
aqi-prediction-airflow-scheduler	—	apache/airflow:3.1.8
aqi-prediction-prometheus	9090	prom/prometheus:latest
aqi-prediction-grafana	3001	grafana/grafana:latest
aqi-prediction-alertmanager	9093	prom/alertmanager:latest

4.3 DVC Pipeline DAG

All large files tracked on DagsHub remote:
https://dagshub.com/astitvashrestha1/E2E_AQI_Prediction_MLOPS

Stage	Command	Outputs
seed_kaggle	python scripts/seed_from_kaggle.py	data/raw/
build_features	python src/features/pipeline.py	data/processed/ data/baseline/
train	docker compose run fastapi python src/train/trainer.py	mlflow/artifacts/
evaluate	docker compose exec fastapi python scripts/evaluate.py	reports/

4.4 Airflow DAG Flow

aqi_hourly_ingest ("5 * * * *")

ingest_openaq + ingest_weather + ingest_aqicn (parallel) → drift_detection → branch → trigger_retrain or no_retrain → log_summary. OpenAQ, weather, and AQICN run simultaneously. Drift detection waits for all three. Branch triggers aqi_retrain DAG via Airflow REST API v2 when drift detected.

aqi_retrain (schedule: None — event-driven)

get_cities → rebuild_features → [retrain_delhi | retrain_mumbai | retrain_kolkata | retrain_chennai | retrain_bengaluru] (all parallel) → reload_api → log_summary. trigger_rule='all_done' on reload_api ensures models reload even if some city training failed.

5. Performance Benchmarks

5.1 Inference Latency

Measured on i7-8750H (6-core, 16GB RAM) running all Docker services simultaneously. Prometheus histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m])) used for P95.

Endpoint	P50 (median)	P95	P99	Notes
GET /forecast/{city}	~1s	~1s	~1s	Includes AQICN + OpenMeteo calls
GET /latest/{city}	~777ms	~1s	~1s	AQICN API call only
GET /drift	~1s	~1s	~1s	KS-test 5 cities parallel
GET /health	~50ms	~95ms	~99ms	In-memory check only
GET /metrics	~50ms	~95ms	~99ms	Prometheus text generation

All endpoints meet the P95 < 10 second acceptance criterion. /forecast P95 ≈ 1s includes live AQICN fetch and OpenMeteo weather forecast API call.

5.2 API Throughput

Measured using Apache Bench (ab) with FastAPI running workers=1 (Prophet not thread-safe). Single-worker constraint limits concurrent throughput.

Test	Requests/sec	Concurrency	Notes
GET /health (no model load)	~422.95 req/s	c=5	Pure in-memory, fast
GET /forecast/Delhi	~0.3 req/s	c=1	Limited by Prophet predict()
GET /latest	~0.6 req/s	c=1	AQICN api call

Throughput bottleneck is Prophet's predict() call (~1s per forecast). Production scaling would use multiple FastAPI replicas with a load balancer, each pre-loading all 5 city models. The workers=1 constraint is per-process — horizontal scaling resolves this.

5.3 Data Engineering Pipeline Speed

Measured on the development machine (i7-8750H, 16GB RAM) running inside Docker with ./data bind-mounted to SSD.

Stage	Data Size	Duration	Notes
OpenAQ ingest (1 city, 2h window)	~50 rows	~15s	API pagination + rate limiting
OpenAQ ingest (5 cities, 2h window)	~250 rows	~120s	Sequential per city
OpenMeteo weather (all cities)	~600 rows	~12s	Parallel fetch possible
Feature engineering (1 city, 3.5 years)	~30,000 rows	~6s	Pandas pipeline (Spark fallback)
Feature engineering (all 5 cities)	~150,000 rows	~31s	Sequential per city
Baseline stats computation	12 monthly files	~3s	Included in pipeline.py
Prophet training (1 city)	~28,000 rows	~4 min	Stan MCMC sampling
Prophet training (all 5 cities)	~140,000 rows	~20 min	Sequential in trainer.py
Drift detection (all 5 cities)	~115 recent rows	~16s	Parallel via ThreadPoolExecutor
Full hourly Airflow DAG	—	~2–3 min	Ingest + drift (no retrain)

The hourly Airflow DAG (ingest + drift detection, no retraining) completes in 2–3 minutes, well within the 60-minute window. Full retraining pipeline takes ~20 minutes and is only triggered on drift detection, not on every hourly run.

Production improvement: OpenAQ ingest for 5 cities could be parallelised using Spark or asyncio, reducing 75s to ~20s. Prophet training could use Ray or Dask for parallel city training.

6. User Interface Screenshots

6.1 Streamlit Dashboard

Figure 3: AQI Forecast Page — city dropdown, current AQI card, 24h chart, health advisory

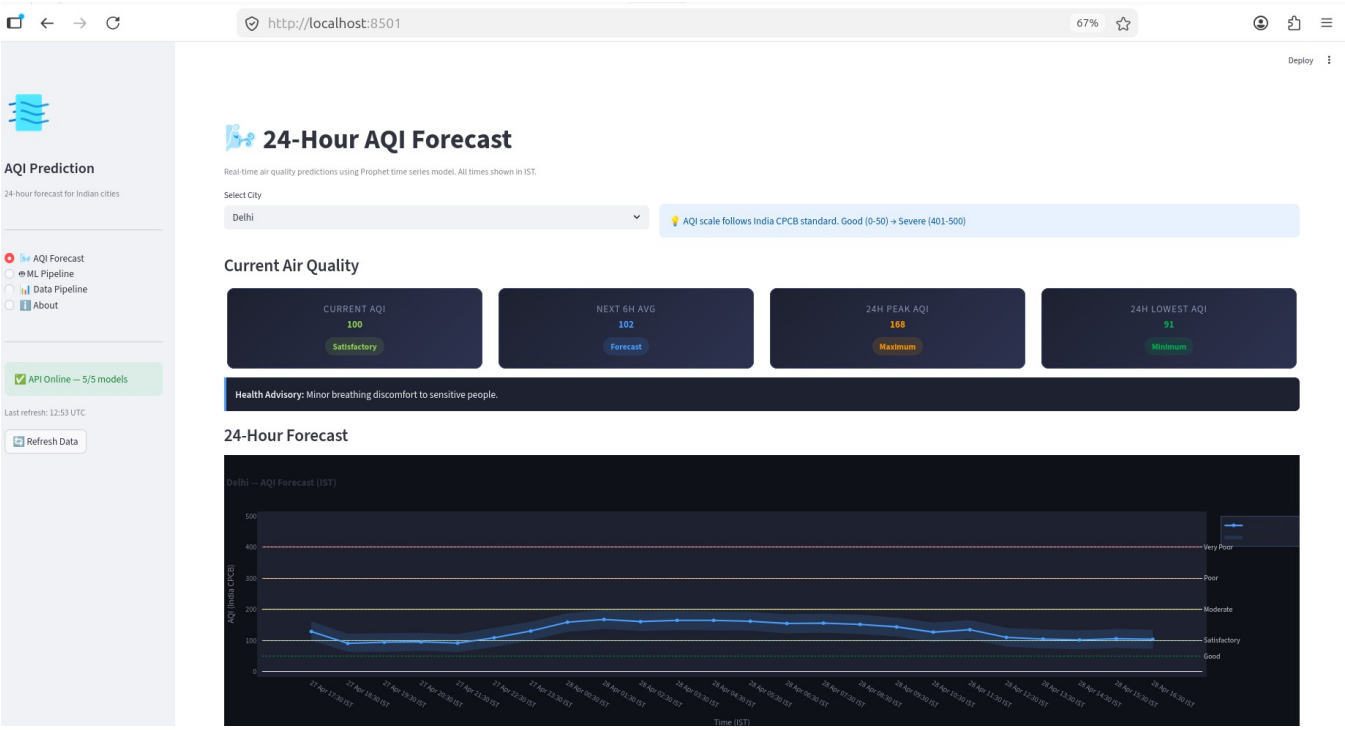
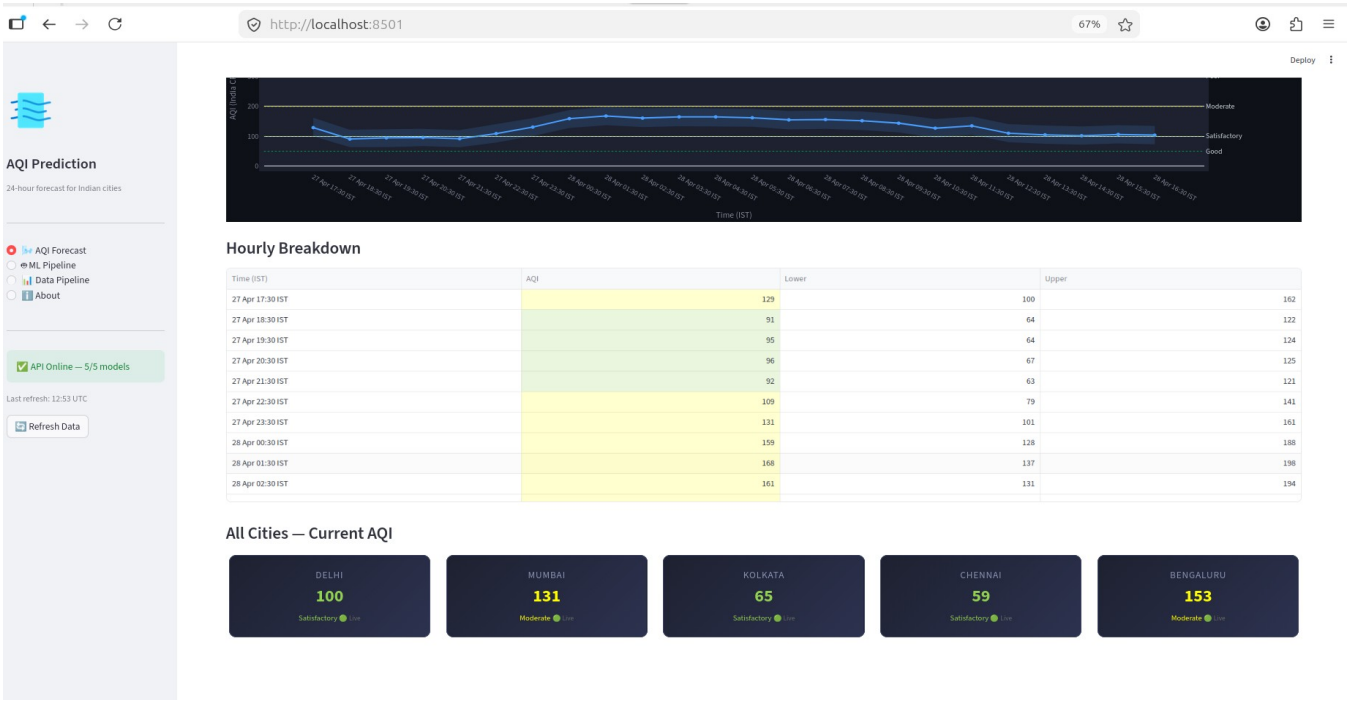
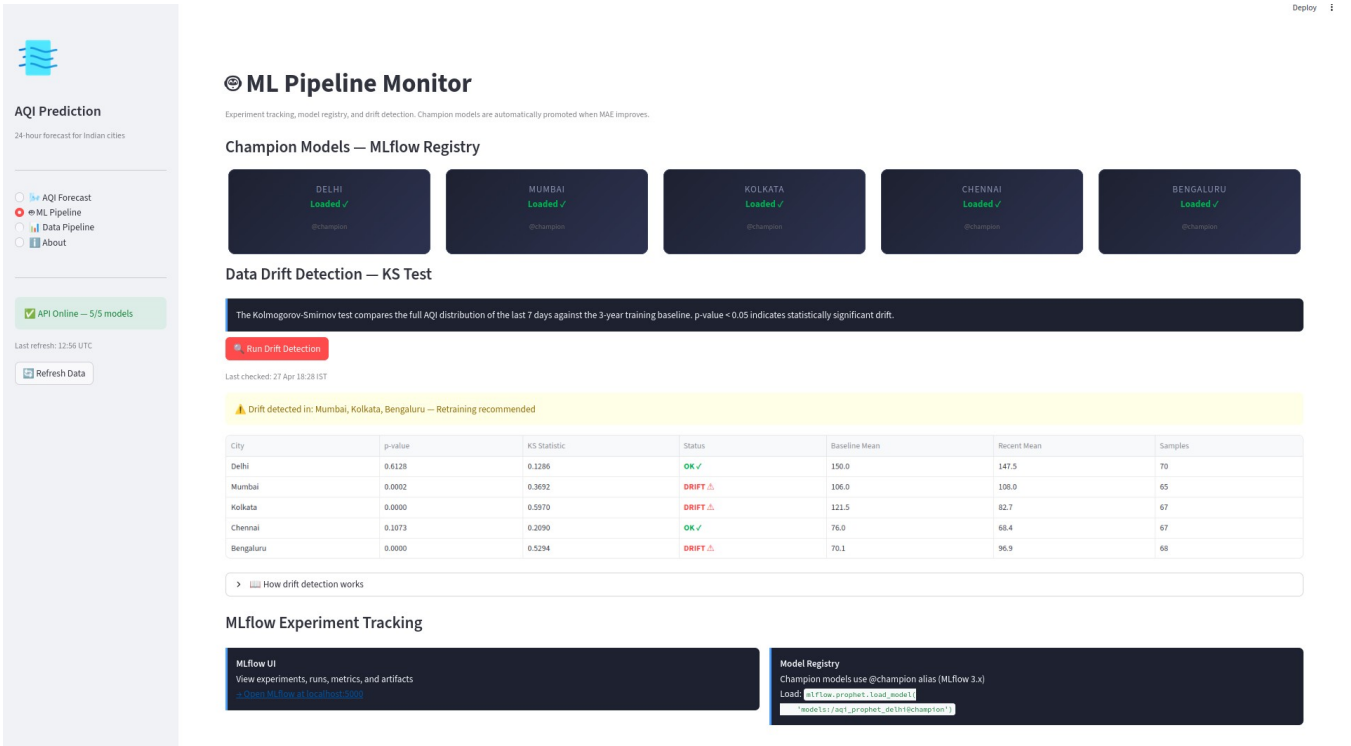


Figure 4: All Cities Summary — current AQI for all 5 cities



6.2 ML Pipeline Monitor

Figure 5: ML Pipeline Page — champion model status, drift detection results



6.3 MLflow Experiment Tracking

Figure 6: MLflow Experiments — runs with MAE, RMSE, MAPE metrics

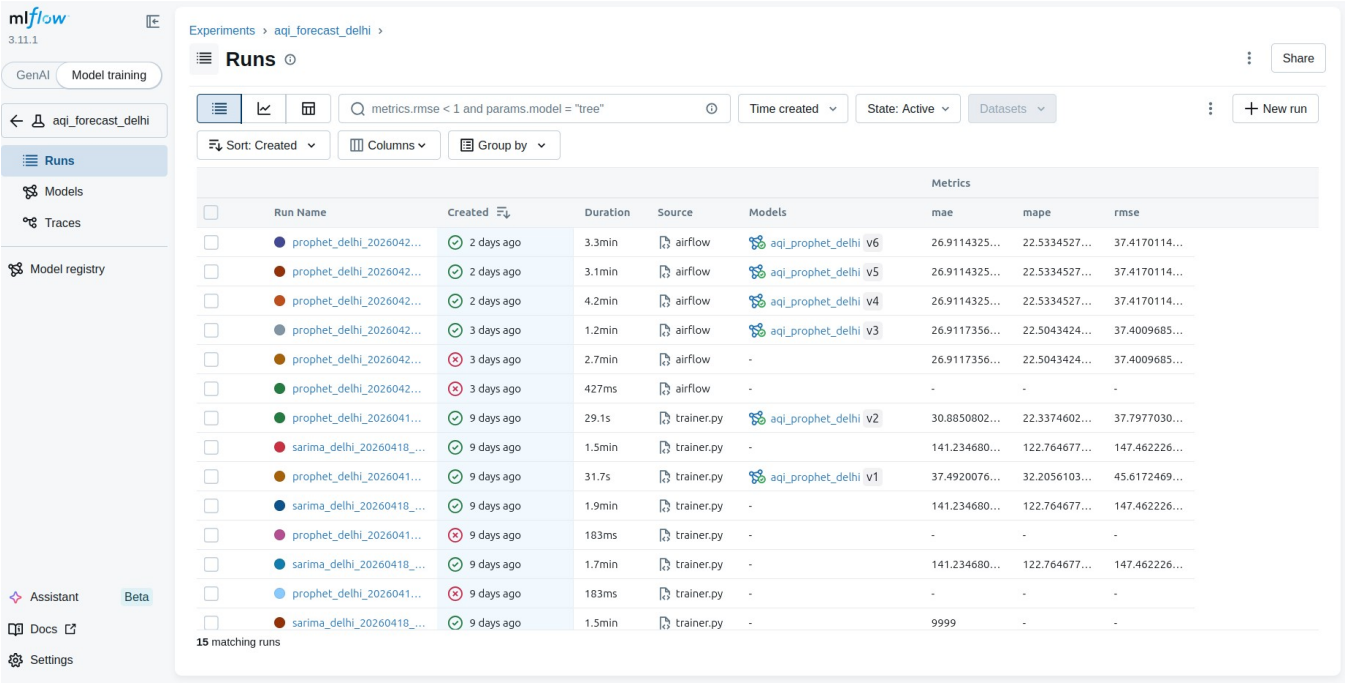
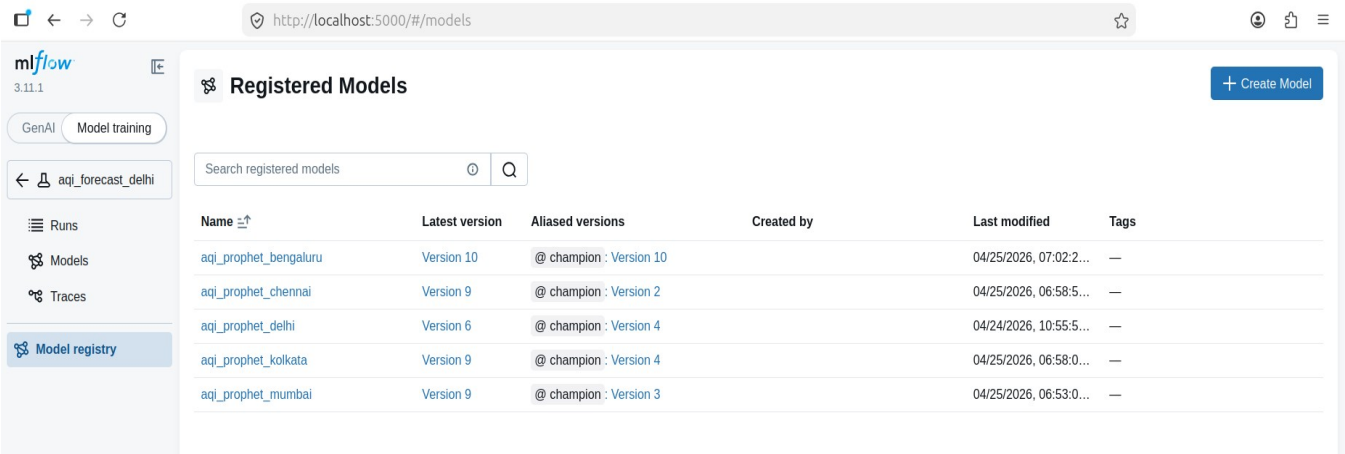


Figure 7: MLflow Model Registry — @champion alias, version history



6.4 Airflow DAG Monitoring

Figure 8: aqi_hourly_ingest DAG — graph view showing parallel ingest tasks

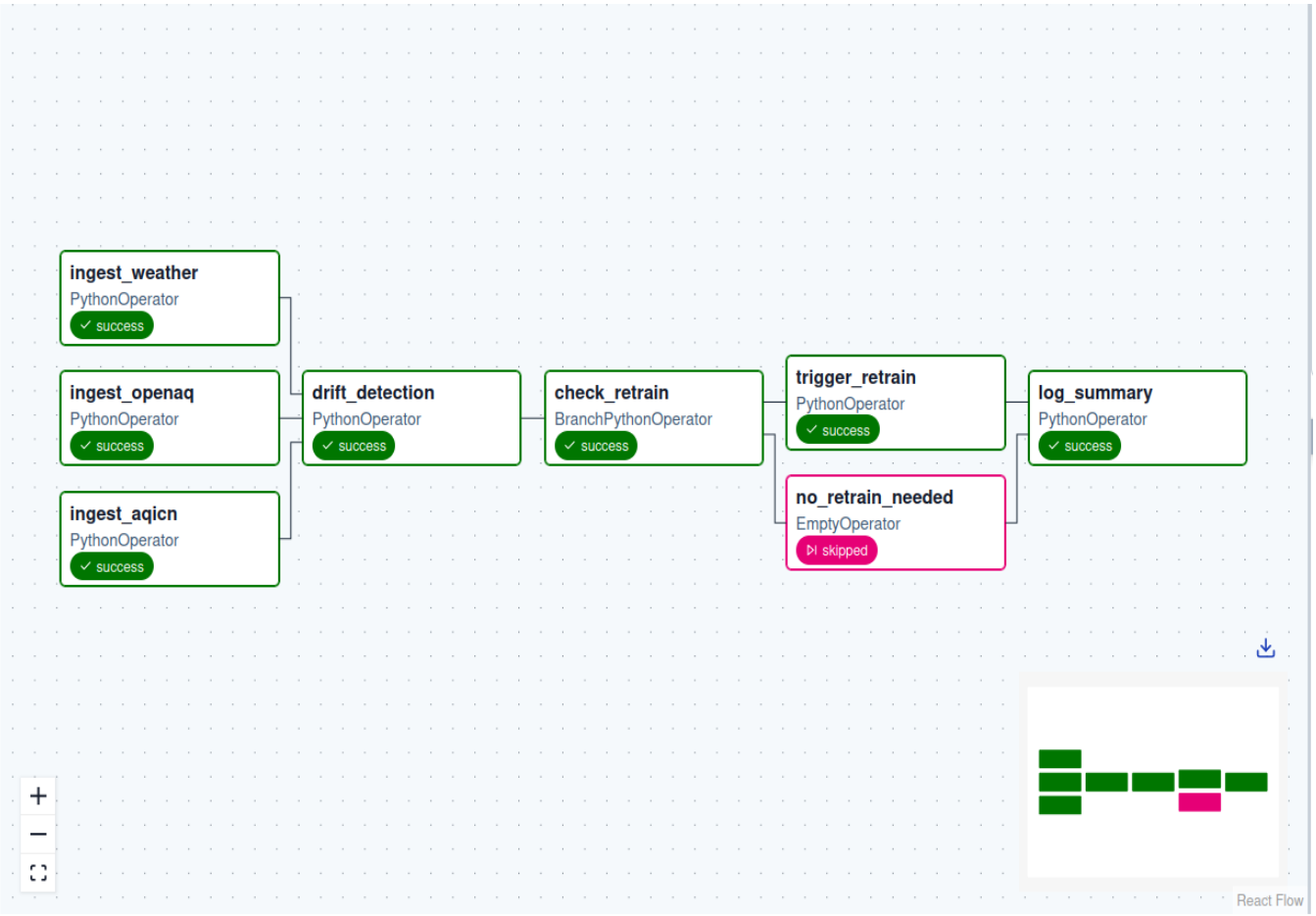
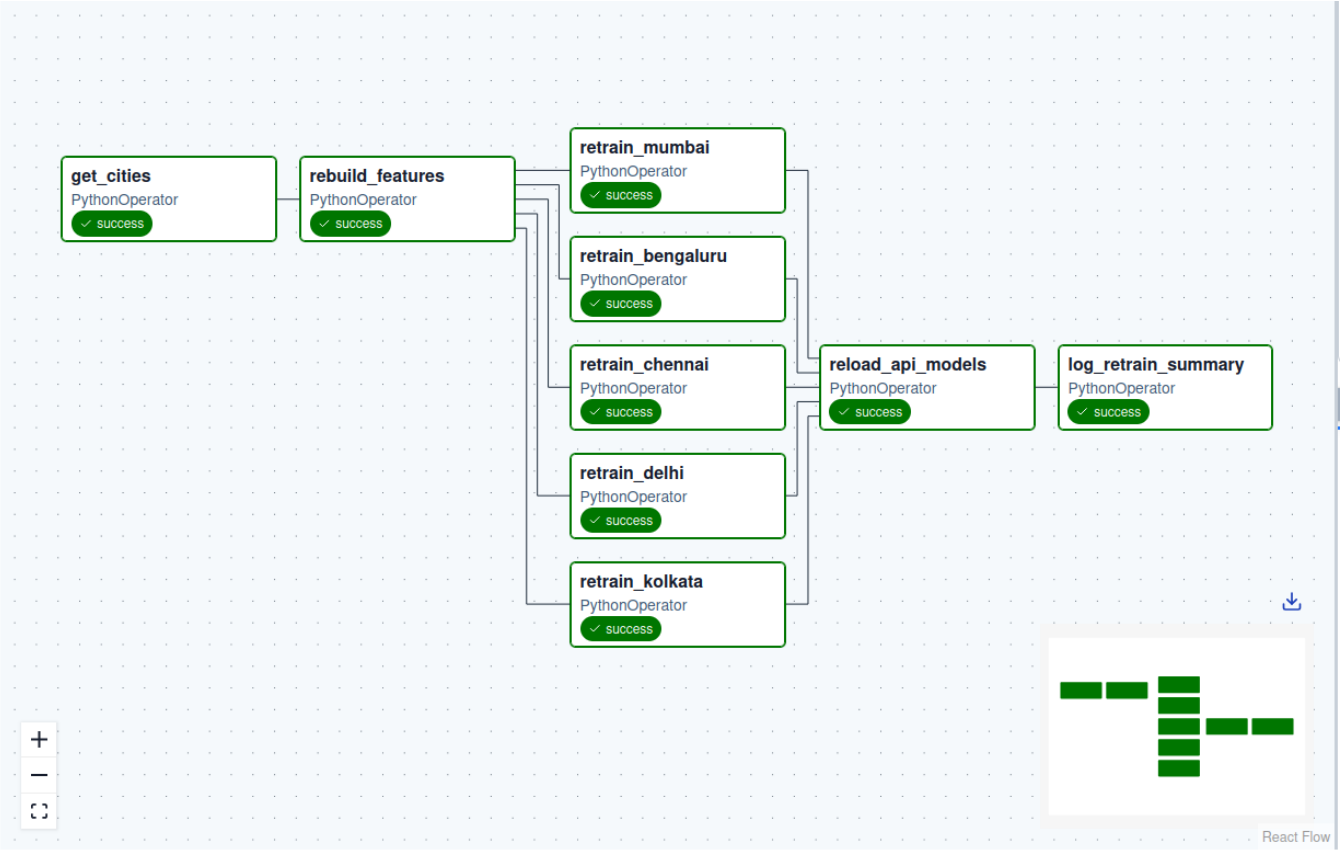
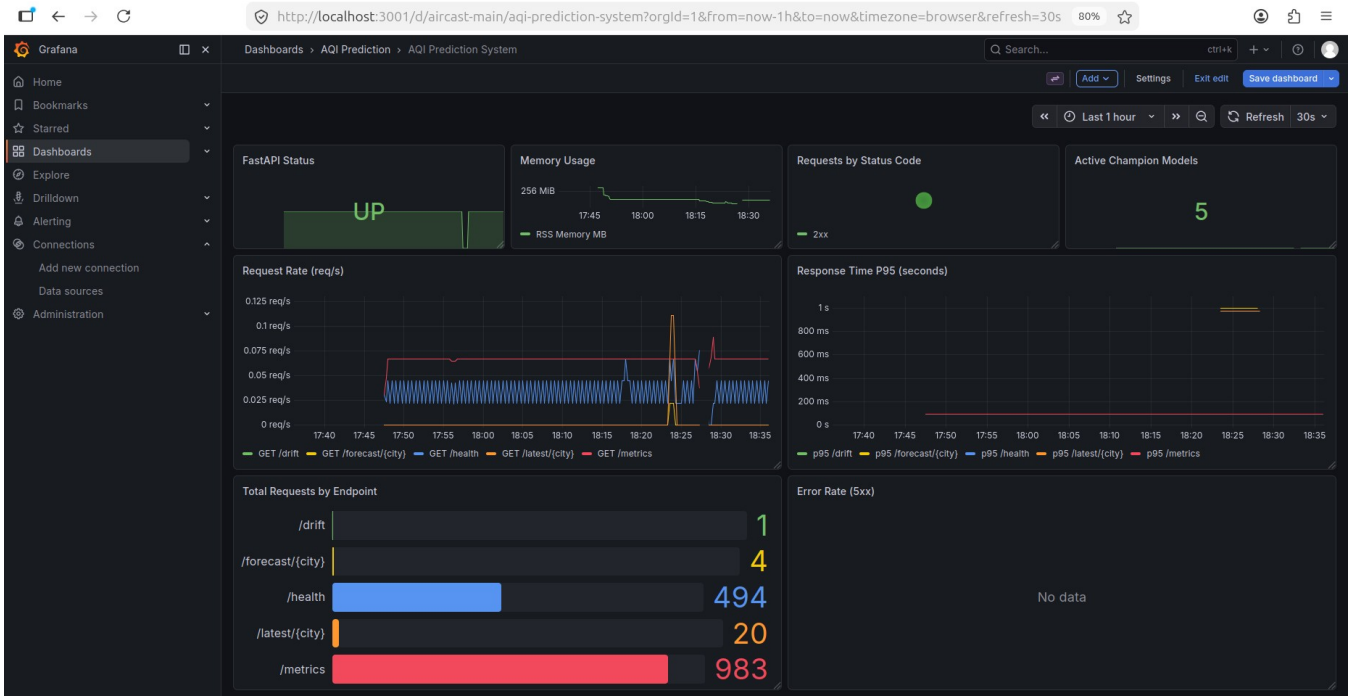


Figure 9: aqi_retrain DAG — parallel city retraining tasks



6.5 Grafana Dashboard

Figure 10: Grafana AQI Prediction Dashboard — 9 panels (status, latency, error rate, throughput)



6.6 Prometheus & AlertManager

Figure 11: Prometheus Alerts — SlowResponseTime, HighErrorRate, HighRequestRate rules

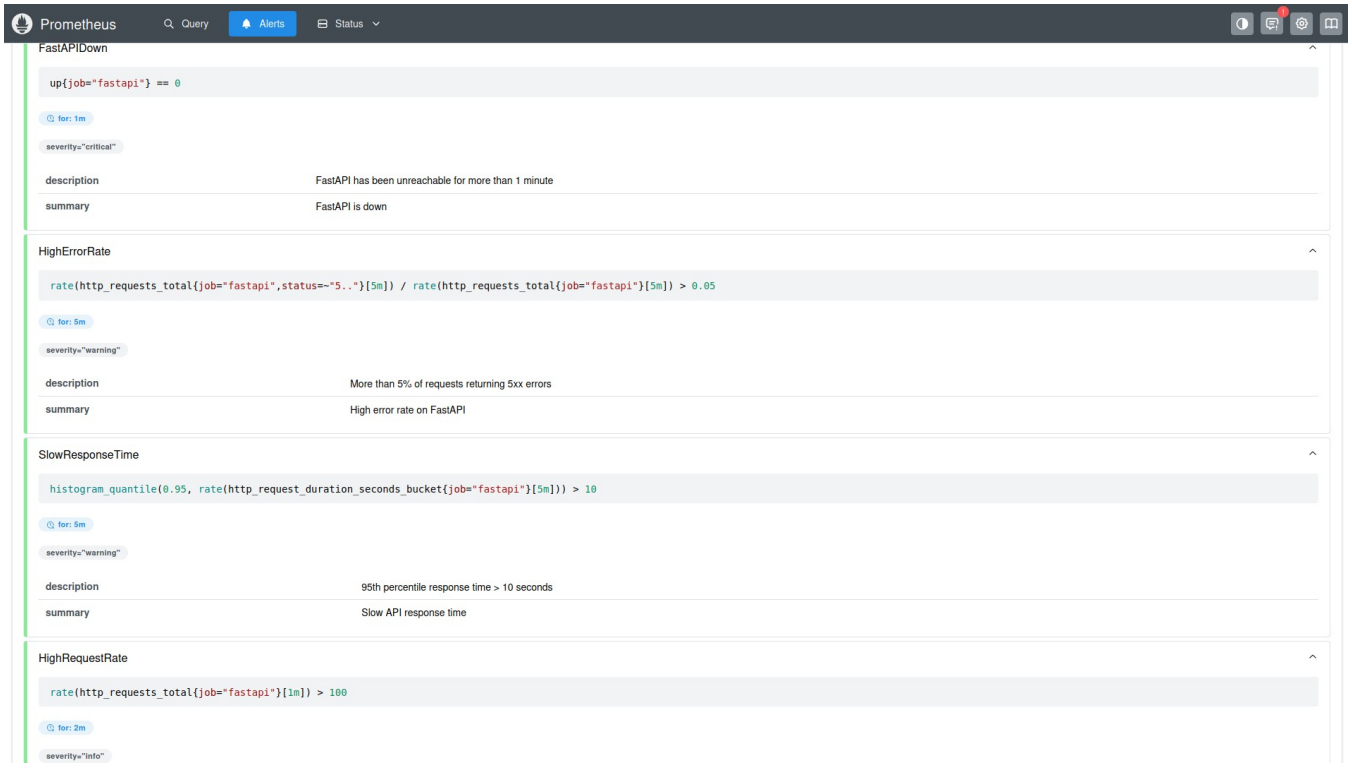
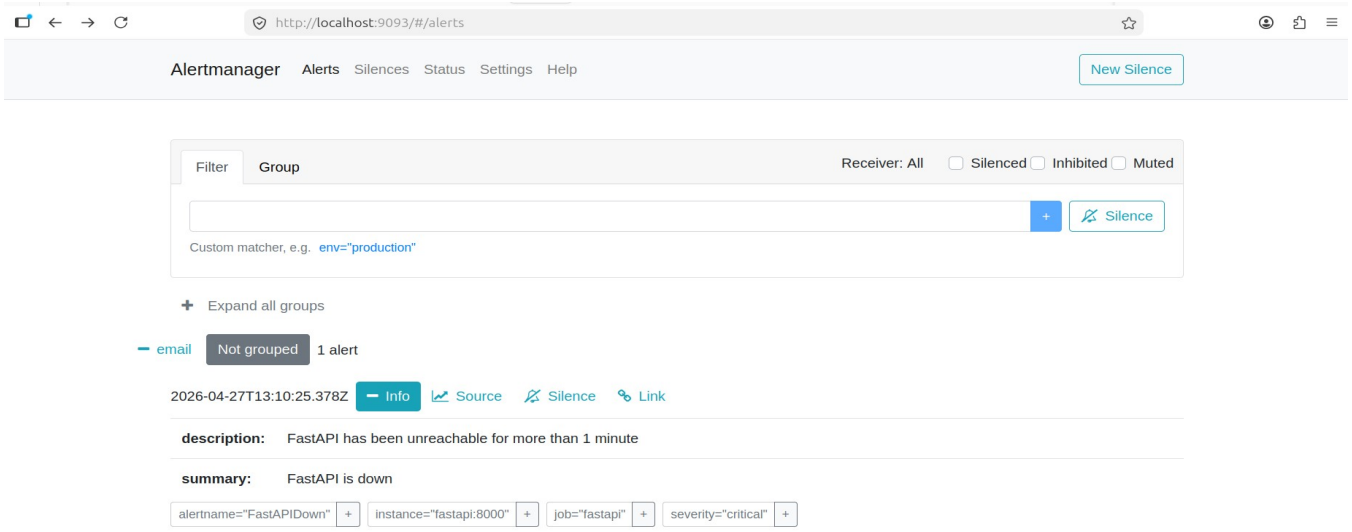
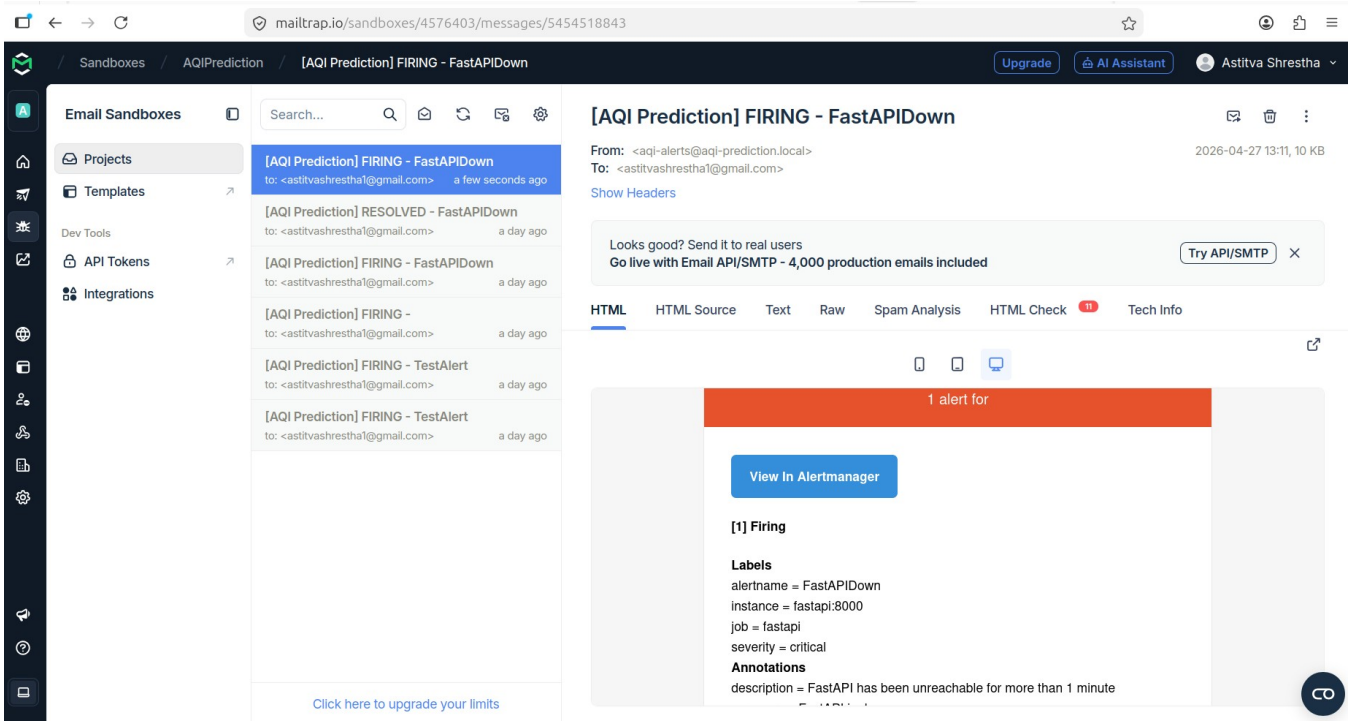


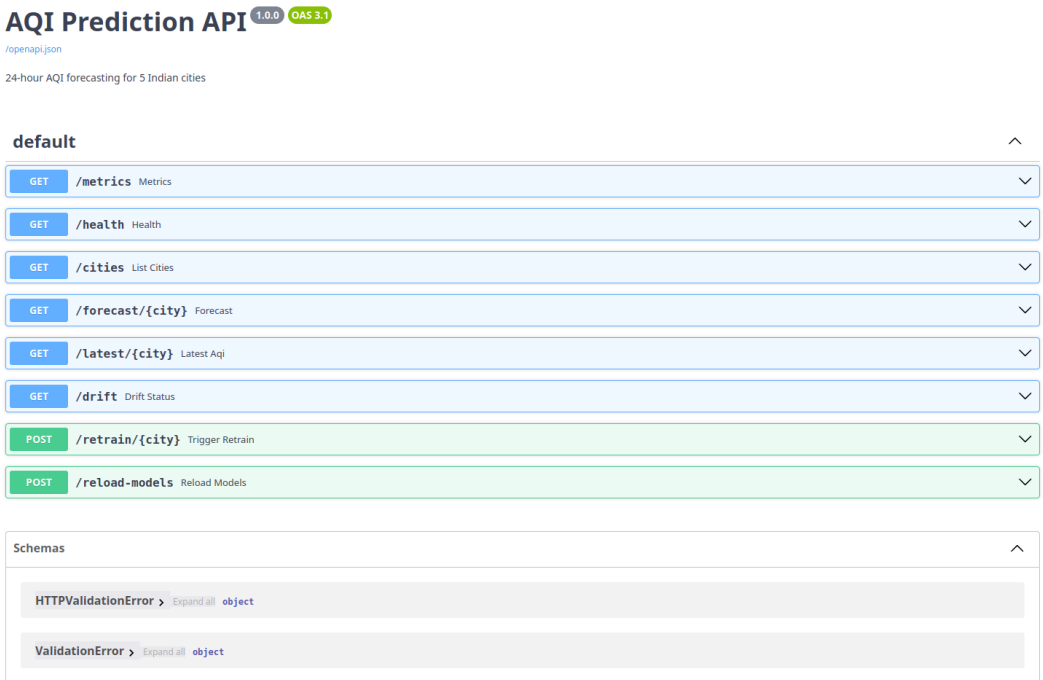
Figure 12: AlertManager — active alerts and routing to Mailtrap





6.7 FastAPI & MLflow

Figure 13: FastAPI /docs — all 8 endpoints



7. Test Plan

7.1 Overview

Test framework: pytest 9.x | File: tests/test_aqi_prediction.py | Total: 72 tests (64 unit + 8 integration)

```
pytest tests/test_aqi_prediction.py -v -m "not integration"  # 64 unit, ~23s
pytest tests/test_aqi_prediction.py -v -m "integration"      # 8 integration
```

Model Evaluation: File: scripts/evaluate.py | Model performance on held-out data

7.2 Test Class Summary

Class	Tests	What is covered
TestHealthEndpoint	6	Status, required fields, timestamp format, model count
TestCitiesEndpoint	6	Response type, all 5 cities present, lat/lon types
TestForecastEndpoint	5	404 for unknown city, case-sensitivity, 503 without model
TestLatestAQIEndpoint	4	404 handling, required response fields
TestDriftDetection	7	KS-test same/different distributions, missing baseline, insufficient data
TestFeaturePipeline	10	crop_burning, festival_period, cyclical ranges, file creation
TestAQICategory	18	All 6 categories, all boundary values (50/51/100/101...), hex color
TestConfig	8	CITIES dict, coordinates within India, Asia/Kolkata timezone
TestIntegration	8	Live /health 5/5 models, /forecast 24h, CI validity, Prometheus
Total	72	64 unit (no server) + 8 integration (requires docker compose up)

Figure 14: pytest output — 72 unit tests passing in ~34s

```
============================= test session starts ==============================
platform linux -- Python 3.11.14, pytest-9.0.3, pluggy-1.6.0 -- /home/astitva/anaconda3/envs/dvc_env/bin/python3.11
cachedir: .pytest_cache
rootdir: /home/astitva/Documents/IITM/MLOps/Assignments/E2E Project/aqi_prediction
configfile: pytest.ini
plugins: asyncio-1.3.0, anyio-4.12.1, hydra-core-1.3.2
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 72 items

tests/test_aqi_prediction.py::TestHealthEndpoint::test_health_returns_200 PASSED [  1%]
tests/test_aqi_prediction.py::TestHealthEndpoint::test_health_response_structure PASSED [  2%]
tests/test_aqi_prediction.py::TestHealthEndpoint::test_health_status_value PASSED [  4%]
tests/test_aqi_prediction.py::TestHealthEndpoint::test_health_timestamp_is_valid_iso PASSED [  5%]
tests/test_aqi_prediction.py::TestHealthEndpoint::test_health_models_fields_are_lists PASSED [  6%]
tests/test_aqi_prediction.py::TestHealthEndpoint::test_health_models_sum_to_five PASSED [  8%]
tests/test_aqi_prediction.py::TestCitiesEndpoint::test_cities_returns_200 PASSED [  9%]
tests/test_aqi_prediction.py::TestCitiesEndpoint::test_cities_returns_dict PASSED [ 11%]
tests/test_aqi_prediction.py::TestCitiesEndpoint::test_cities_has_five_entries PASSED [ 12%]
tests/test_aqi_prediction.py::TestCitiesEndpoint::test_cities_contains_expected PASSED [ 13%]
tests/test_aqi_prediction.py::TestCitiesEndpoint::test_cities_have_required_fields PASSED [ 15%]
tests/test_aqi_prediction.py::TestCitiesEndpoint::test_cities_coordinates_are_numbers PASSED [ 16%]
tests/test_aqi_prediction.py::TestForecastEndpoint::test_forecast_invalid_city_returns_404 PASSED [ 18%]
tests/test_aqi_prediction.py::TestForecastEndpoint::test_forecast_city_case_sensitive PASSED [ 19%]
tests/test_aqi_prediction.py::TestForecastEndpoint::test_forecast_all_cities_not_404 PASSED [ 20%]
tests/test_aqi_prediction.py::TestForecastEndpoint::test_forecast_no_model_returns_503 PASSED [ 22%]
tests/test_aqi_prediction.py::TestForecastEndpoint::test_forecast_response_structure_when_loaded PASSED [ 23%]
tests/test_aqi_prediction.py::TestLatestAQIEndpoint::test_latest_invalid_city_returns_404 PASSED [ 25%]
tests/test_aqi_prediction.py::TestLatestAQIEndpoint::test_latest_valid_city_not_404 PASSED [ 26%]
tests/test_aqi_prediction.py::TestLatestAQIEndpoint::test_latest_response_has_required_fields PASSED [ 27%]
tests/test_aqi_prediction.py::TestLatestAQIEndpoint::test_latest_all_cities_not_404 PASSED [ 29%]
tests/test_aqi_prediction.py::TestDriftDetection::test_no_drift_same_distribution PASSED [ 30%]
tests/test_aqi_prediction.py::TestDriftDetection::test_drift_different_distribution PASSED [ 31%]
tests/test_aqi_prediction.py::TestDriftDetection::test_drift_p_value_range PASSED [ 33%]
tests/test_aqi_prediction.py::TestDriftDetection::test_drift_check_city_with_mock PASSED [ 34%]
tests/test_aqi_prediction.py::TestDriftDetection::test_drift_result_types PASSED [ 36%]
tests/test_aqi_prediction.py::TestDriftDetection::test_drift_handles_missing_baseline PASSED [ 37%]
tests/test_aqi_prediction.py::TestDriftDetection::test_drift_handles_insufficient_recent_data PASSED [ 38%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_bonus_features_crop_burning_october PASSED [ 40%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_bonus_features_crop_burning_july PASSED [ 41%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_bonus_features_festival_october PASSED [ 43%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_bonus_features_festival_march PASSED [ 44%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_features_sorted_by_time PASSED [ 47%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_required_features_exist PASSED [ 48%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_no_negative_aqi PASSED [ 50%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_save_features_creates_file PASSED [ 51%]
tests/test_aqi_prediction.py::TestFeaturePipeline::test_compute_baseline_stats_creates_file PASSED [ 52%]
tests/test_aqi_prediction.py::TestAQICategory::test_good_category PASSED [ 54%]
tests/test_aqi_prediction.py::TestAQICategory::test_satisfactory_category PASSED [ 55%]
tests/test_aqi_prediction.py::TestAQICategory::test_moderate_category PASSED [ 56%]
tests/test_aqi_prediction.py::TestAQICategory::test_poor_category PASSED [ 58%]
tests/test_aqi_prediction.py::TestAQICategory::test_very_poor_category PASSED [ 59%]
tests/test_aqi_prediction.py::TestAQICategory::test_severe_category PASSED [ 61%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_50_is_good PASSED [ 62%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_51_is_satisfactory PASSED [ 63%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_100_is_satisfactory PASSED [ 65%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_101_is_moderate PASSED [ 66%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_200_is_moderate PASSED [ 68%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_201_is_poor PASSED [ 69%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_300_is_poor PASSED [ 70%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_301_is_very_poor PASSED [ 72%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_400_is_very_poor PASSED [ 73%]
tests/test_aqi_prediction.py::TestAQICategory::test_boundary_401_is_severe PASSED [ 75%]
tests/test_aqi_prediction.py::TestAQICategory::test_category_has_label_color_advice PASSED [ 76%]
tests/test_aqi_prediction.py::TestAQICategory::test_category_color_is_hex PASSED [ 77%]
tests/test_aqi_prediction.py::TestConfig::test_cities_dict_exists PASSED [ 79%]
tests/test_aqi_prediction.py::TestConfig::test_cities_has_five_entries PASSED [ 80%]
tests/test_aqi_prediction.py::TestConfig::test_all_expected_cities_present PASSED [ 81%]
tests/test_aqi_prediction.py::TestConfig::test_all_cities_have_lat_lon PASSED [ 83%]
tests/test_aqi_prediction.py::TestConfig::test_all_cities_have_timezone PASSED [ 84%]
tests/test_aqi_prediction.py::TestConfig::test_all_cities_have_openaq_ids PASSED [ 86%]
tests/test_aqi_prediction.py::TestConfig::test_coordinates_within_india PASSED [ 87%]
tests/test_aqi_prediction.py::TestConfig::test_delhi_coordinates_approximate PASSED [ 88%]
tests/test_aqi_prediction.py::TestIntegration::test_api_health_live PASSED [ 90%]
tests/test_aqi_prediction.py::TestIntegration::test_forecast_delhi_live PASSED [ 91%]
tests/test_aqi_prediction.py::TestIntegration::test_forecast_aqi_values_in_valid_range PASSED [ 93%]
tests/test_aqi_prediction.py::TestIntegration::test_forecast_confidence_intervals_valid PASSED [ 94%]
tests/test_aqi_prediction.py::TestIntegration::test_all_five_cities_forecast_live PASSED [ 95%]
tests/test_aqi_prediction.py::TestIntegration::test_metrics_endpoint_has_http_requests_total PASSED [ 97%]
tests/test_aqi_prediction.py::TestIntegration::test_latest_aqi_returns_valid_value PASSED [ 98%]
tests/test_aqi_prediction.py::TestIntegration::test_forecast_category_labels_valid PASSED [100%]

============================== 72 passed in 33.89s ==============================
```

7.3 Acceptance Criteria Results (Model Evaluation)

City	Target	MAE	RMSE	Status
Delhi	MAE ≤ 50	27.56	39.23	✓ PASS
Mumbai	MAE ≤ 50	17.03	35.01	✓ PASS
Kolkata	MAE ≤ 50	19.07	32.50	✓ PASS
Chennai	MAE ≤ 50	19.10	32.61	✓ PASS
Bengaluru	MAE ≤ 50	17.54	27.01	✓ PASS

ALL 5 CITIES PASS MAE ≤ 50 AQI acceptance criterion

```
======================================================================================
AQI PREDICTION SYSTEM — MODEL EVALUATION REPORT
Generated: 2026-04-27 13:21 UTC
======================================================================================

City              MAE     RMSE     MAPE    Coverage    Cat Acc    Bnd Err    Rush MAE
--------------------------------------------------------------------------------------
Delhi           27.56    39.23   23.45%       63.4%      71.5%      10.6%       26.24
Mumbai          17.03    35.01   20.69%       78.0%      78.3%       9.9%       17.92
Kolkata         19.07    32.50   21.87%       79.5%      80.7%       7.1%       18.99
Chennai         19.10    32.61   26.88%       63.5%      61.4%      21.1%       17.95
Bengaluru       17.54    27.01   17.43%       69.2%      55.4%      25.4%       18.29
--------------------------------------------------------------------------------------
Average         20.06    33.27   22.06%       70.7%
======================================================================================

Metric definitions:
  MAE      — Mean Absolute Error (AQI points)
  RMSE     — Root Mean Squared Error (penalises large errors)
  MAPE     — Mean Absolute Percentage Error (%)
  Coverage — % of true values within 95% confidence interval
  Cat Acc  — % of hours where predicted AQI category matches actual
  Bnd Err  — % category errors where actual AQI is near a threshold
  Rush MAE — MAE during morning/evening rush hours (7-10, 17-20)

Calibration by interval-width bins (target coverage ~95%):
  Delhi        B1=64.9%, B2=66.2%, B3=57.3%, B4=63.8%, B5=64.9%
  Mumbai       B1=83.1%, B2=77.2%, B3=76.4%, B4=75.6%, B5=77.4%
  Kolkata      B1=75.6%, B2=77.2%, B3=79.7%, B4=87.8%, B5=77.4%
  Chennai      B1=60.5%, B2=68.3%, B3=59.3%, B4=64.2%, B5=65.3%
  Bengaluru    B1=70.2%, B2=73.2%, B3=63.4%, B4=69.9%, B5=69.4%

Compact summary scores:
  calibration_gap = 95 - coverage
  boundary_share_of_errors = boundary_error_rate / (100 - category_accuracy)
  Delhi        calibration_gap=31.6% | boundary_share_of_errors=0.37 | category_accuracy_ex_boundary=77.8% | coverage_95_ex_boundary=65.1%
  Mumbai       calibration_gap=17.0% | boundary_share_of_errors=0.46 | category_accuracy_ex_boundary=82.6% | coverage_95_ex_boundary=77.1%
  Kolkata      calibration_gap=15.5% | boundary_share_of_errors=0.37 | category_accuracy_ex_boundary=83.8% | coverage_95_ex_boundary=82.5%
  Chennai      calibration_gap=31.5% | boundary_share_of_errors=0.55 | category_accuracy_ex_boundary=69.5% | coverage_95_ex_boundary=59.6%
  Bengaluru    calibration_gap=25.8% | boundary_share_of_errors=0.57 | category_accuracy_ex_boundary=62.2% | coverage_95_ex_boundary=51.6%

Acceptance Criteria:
  Delhi        MAE ≤ 50:  27.56 ✓ PASS
  Mumbai       MAE ≤ 50:  17.03 ✓ PASS
  Kolkata      MAE ≤ 50:  19.07 ✓ PASS
  Chennai      MAE ≤ 50:  19.10 ✓ PASS
  Bengaluru    MAE ≤ 50:  17.54 ✓ PASS

Overall: ALL TESTS PASSED ✓
======================================================================
2026-04-27 13:21:11 INFO Report saved: /app/reports/evaluation_20260427_132111.json
2026-04-27 13:21:11 INFO Report saved: /app/reports/evaluation_20260427_132111.csv
2026-04-27 13:21:11 INFO Latest:       /app/reports/evaluation_latest.json
```

8. User Manual

8.1 Getting Started

Open http://localhost:8501 in your browser. No login required. Data auto-refreshes every 5 minutes. Click Refresh Data in the sidebar for immediate refresh.

8.2 AQI Scale (India CPCB Standard)

Category	AQI Range	Health Impact	Recommended Action
Good	0–50	No health implications	Ideal for outdoor activities
Satisfactory	51–100	Minor discomfort for sensitive people	Generally safe
Moderate	101–200	Breathing discomfort (lung/heart patients)	Sensitive groups limit outdoor exposure
Poor	201–300	Breathing discomfort for most people	Avoid prolonged outdoor exposure
Very Poor	301–400	Respiratory illness on prolonged exposure	Stay indoors
Severe	401–500	Serious health impact	Avoid all outdoor activity

8.3 Page-by-Page Guide

Page 1 — AQI Forecast

Select a city from the dropdown. Current AQI card shows live reading from AQICN. Health Advisory shows recommended actions. 24-hour chart shows predicted AQI (blue line) with 95% confidence interval (shaded area). Scroll down for all-cities summary.

Page 2 — ML Pipeline Monitor

Shows champion model status per city (Loaded/Missing). Run Drift Detection button triggers KS-test for all 5 cities and shows p-values, KS statistics, baseline mean vs recent mean. Toggle ?seasonal=true for same-month comparison.

Page 3 — Data Pipeline

Shows data ingestion architecture, source descriptions, pipeline status, and links to Airflow (localhost:8080), Grafana (localhost:3001), and Prometheus (localhost:9090).

Page 4 — About

System information, technology stack, AQI scale reference, and user manual.

8.4 Service URLs

Service	URL	Credentials
Dashboard	http://localhost:8501	None required
API Docs (Swagger)	http://localhost:8000/docs	None required
MLflow UI	http://localhost:5000	None required
Airflow UI	http://localhost:8080	admin / admin
Grafana	http://localhost:3001	admin / admin
Prometheus	http://localhost:9090	None required
AlertManager	http://localhost:9093	None required

8.5 Troubleshooting

Problem	Solution
API Offline in sidebar	Run: bash scripts/start_demo.sh
0/5 models warning	Run trainer inside Docker: bash scripts/retrain.sh
Forecast shows stale times	Click Refresh Data button in sidebar
Drift detection hangs	Normal — KS-test takes 30–60s for 5 cities
City shows N/A for current AQI	AQICN API unavailable — disk cache used automatically
No alert emails received	Check localhost:9093, verify MAILTRAP_* in .env

9. Known Limitations and Production Improvements

Limitation	Current State	Production Fix
Coverage 70% vs 95% target	Prophet CI underestimated — fat-tailed AQI distributions	Conformal prediction intervals
Seasonal drift false positives	Mitigated with ?seasonal=true mode	Same-season stratified comparison
Prophet not thread-safe	workers=1 in FastAPI (throughput ~1 req/s)	Multiple replicas behind load balancer
SARIMA memory 4–6GB	Taken high memory	Distributed training with Ray
OpenAQ ingest sequential	~75s for 5 cities	Async/parallel ingest
Prophet training sequential	~20 min for 5 cities	Parallel training with Ray
KS-test sample size bias	Fixed: baseline subsampled to recent N	Bootstrap KS-test for robustness
AQICN live inference	Picks a station at random	Take median of some stations within a city for a more accurate reading